Internship at CAVE LABS-PESU

"3D Evidence Reconstruction on Android using 2D Videos"

Submitted by:

Aditya Narayan Dandia	PES1UG24AM017

Under the guidance of

Dr. Adithya Balasubramanyam
Professor,
PES University

1st June 2026 - 27th July 2026 (8 Weeks)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

FACULTY OF ENGINEERING

PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)

100ft Ring Road, Bengaluru – 560 085, Karnataka, India

DECLARATION

I, Aditya Narayan Dandia, hereby declare that the project entitled "3D Evidence Reconstruction on Android using 2D Videos" has been carried out at CAVE LABS-PESU by me under the guidance of Dr. Adithya Balasubramanyam, Professor, PES University, and submitted in partial fulfillment of the credits for the degree of Bachelor of Technology in Computer Science and Engineering of PES University, Bengaluru during the academic semester 1st June 2026 - 27th July 2026. The matter embodied in this report has not been submitted to any other university or institution for the award of any degree.

Aditya Narayan Dandia	**PES1UG24AM017**

ACKNOWLEDGEMENT

I would like to express my gratitude to my guide Dr. Adithya Balasubramanyam, Professor, CAVE LABS-PESU, for his continuous guidance, assistance and encouragement throughout the development of this project.

I am grateful to the internship coordinator Indu Radhakrishnan, Dept. of Computer Science and Engineering, PES University, for organizing, managing and helping out with the entire process.

I take this opportunity to thank Dr. Shylaja S S, Chairperson, Department of Computer Science and Engineering, PES University, for all the knowledge and support I have received from the department.

I am deeply grateful to Prof. Jawahar Doreswamy, Chancellor, PES University, and Dr. Suryaprasad J, Vice-Chancellor, PES University, for providing me various opportunities and enlightenment every step of the way.

Finally, this internship could not have been completed without the continual support and encouragement I have received from my parents and my friends.

ABSTRACT

3D Evidence Reconstruction on Android using 2D Videos is a standalone 3D scene reconstruction pipeline developed during an eight-week internship at CAVE LABS-PESU, targeting real-world capture-to-render workflows using 3D Gaussian Splatting. The system consists of an Android capture application built with Kotlin and Jetpack Compose, and a FastAPI backend that orchestrates a multi-stage reconstruction pipeline: camera-pose estimation via COLMAP, splat training via FastGS, custom appearance-based cleanup filtering, quality and calibration metrics, and compressed splat delivery for in-app rendering.

Over the course of the internship, the author was responsible for the majority of the backend pipeline: scaffolding the FastAPI service, integrating COLMAP for structure-from-motion and camera pose recovery, integrating and stabilizing FastGS training, designing a cleanup module using local-neighborhood color-consistency filtering and quaternion rotation-sanity checks to remove floating/streaking Gaussian artifacts, and building quality-assessment tooling combining COLMAP reprojection-error statistics, ArUco-marker-based physical scale calibration via DLT triangulation, and reference-based metrics (PSNR, SSIM, LPIPS). The author also implemented the Android-to-backend networking layer and contributed to an early AR Foundation-based capture prototype, which was later superseded by a more robust CameraX-based capture flow.

The result is an end-to-end pipeline capable of taking a short handheld video capture and producing a cleaned, compressed, quantitatively-assessed 3D Gaussian Splat scene, viewable directly on-device. This report documents the project's design, implementation, and outcomes.

TABLE OF CONTENTS

1. Introduction 1
1.1 About CAVE LABS-PESU 1
1.2 Internship Project Details 1
2. Project Abstract and Scope 2
3. Project Design Details and Technologies Used 3
4. Coding / Implementation Details 5
5. Project Results / Learning Outcomes 10
6. Conclusion 12
7. References / Bibliography 13

LIST OF TABLES AND FIGURES

List of Tables

Table 1. Pipeline Stages, Components, and Tools 3

List of Figures

Figure 1. 3D Evidence Reconstruction on Android using 2D Videos System Architecture ... 3
Figure 2. SplatStudio Android App – Capture Screen 6
Figure 3. SplatStudio Android App – Progress Bar 6
Figure 4. SplatStudio Android App – Rendered View 7
Figure 5. SplatStudio Android App – Audit Log 7
Figure 6. SplatStudio Android App – Metrics 8
Figure 7. SplatStudio Android App – Server Page 9

1. Introduction

1.1 About CAVE LABS-PESU

CAVE (CAVE Augmented and Virtual Environments) Lab is a research initiative under the Center for IoT, PES University, that focuses on exploring various applications of mixed reality, digital twin, human motion tracking, and generative alternate realities in the metaverse. The goal of the lab is to teach, learn, research, innovate, and collaborate with industry in the mixed reality space. The lab is equipped with hardware that enables research in the mixed reality space. CAVE also focuses on the use of VR technology in remote healthcare, neurodevelopmental disorders, and related domains.

1.2 Internship Project Details

Project Title: "3D Evidence Reconstruction on Android using 2D Videos"

Is it part of a bigger project: No — 3D Evidence Reconstruction on Android using 2D Videos was undertaken as a standalone internship project at CAVE LABS-PESU.

Role and Responsibilities: As part of the project team, the author's primary responsibility was building and stabilizing the backend reconstruction pipeline — from raw video capture to a cleaned, quality-assessed, compressed **3D Gaussian Splat**. This included integrating **COLMAP** for camera pose recovery, integrating and stabilizing **FastGS** for splat training, designing and implementing the appearance-based cleanup filters, and building the quality and calibration metrics module. The author also implemented the Android application's networking layer connecting the capture app to the backend.

Day-to-Day Workflow: Each day (or the evening before), the team met to discuss the day's to-do list and picked three high-priority items to work through, with a daily stand-up meeting involving the whole team. Work was assigned on a weekly basis with weekly targets and reviews, with tasks divided either by existing skill or handed to whoever wanted to learn something new — the pipeline stages themselves were self-organized by the team rather than top-down assigned. Progress and completed work were reviewed through these stand-ups and periodic demos.

2. Project Abstract and Scope

3D Evidence Reconstruction on Android using 2D Videos aims to make high-fidelity 3D scene reconstruction accessible from a simple handheld video capture, using **3D Gaussian Splat**ting as the underlying scene representation. The scope of the project spans the full capture-to-render pipeline: an Android application for video capture, a **FastAPI** backend that performs **Structure-from-Motion** (via **COLMAP**) and **Gaussian Splat** training (via **FastGS**), a cleanup stage that removes visual artifacts introduced during reconstruction, a metrics stage that quantitatively evaluates reconstruction quality and physical scale accuracy, and a compression/serving stage that delivers the final splat for on-device viewing.

The project's scope was intentionally end-to-end rather than limited to a single research component, reflecting CAVE LABS-PESU's emphasis on deployable pipelines. Within this scope, the author's individual contribution centered on the backend reconstruction and cleanup pipeline, and the Android-to-backend networking integration.

3. Project Design Details and Technologies Used

3D Evidence Reconstruction on Android using 2D Videos is structured as a client-server pipeline. The Android application (built with **Kotlin** and **Jetpack Compose**) handles video capture and communicates with the backend over a REST API implemented with **Retrofit**, **OkHttp**, and **Moshi** for multipart video upload and asynchronous job polling. The **FastAPI** backend orchestrates the reconstruction pipeline as a background task: **COLMAP** performs feature extraction, sequential feature matching, sparse mapping, and image undistortion to recover camera poses; **FastGS** then trains a **3D Gaussian Splat** representation from the posed images.

Following training, a custom cleanup module filters the raw splat output using local-neighborhood color-consistency checks on the spherical-harmonics DC (base color) term, combined with a quaternion rotation-sanity filter, to remove floating and streaking Gaussian artifacts. A metrics module then evaluates the cleaned reconstruction using **COLMAP** reprojection-error statistics, **ArUco**-marker-based physical scale calibration via Direct Linear Transformation (DLT) triangulation, and reference-based image quality metrics (**PSNR**, **SSIM**, **LPIPS**) against held-out renders. The final splat is compressed from .ply to the .sog format via splat-transform and served to the Android app through a /api/download endpoint, where it is rendered on-device.

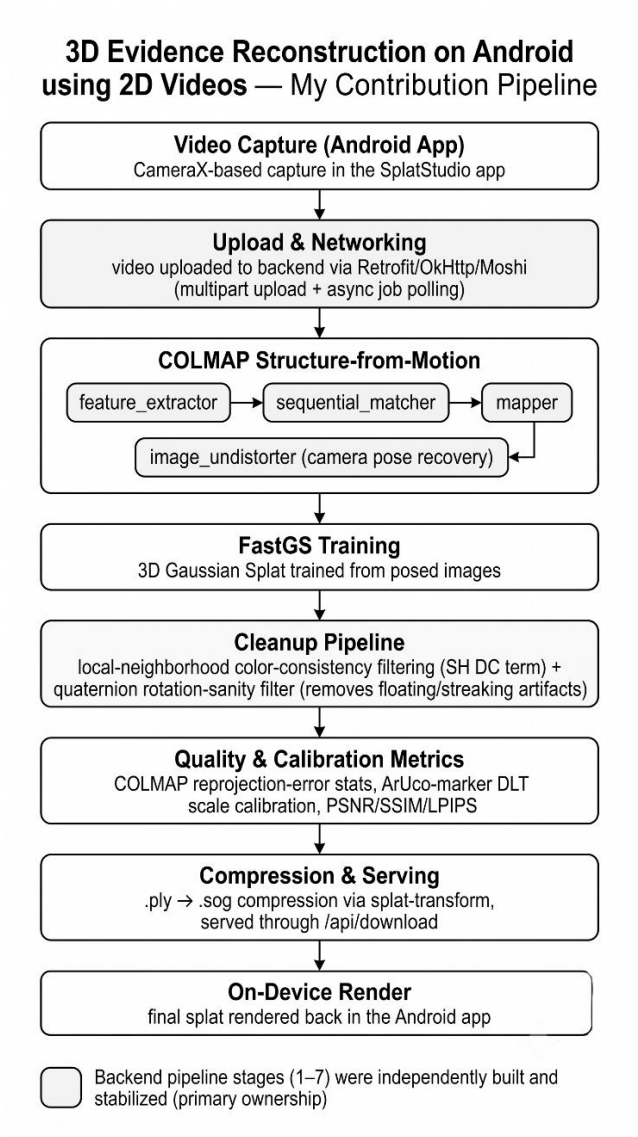

Figure 1: 3D Evidence Reconstruction on Android using 2D Videos System Architecture(My contributions)

Table 1 below summarizes the major pipeline stages, the components/files implementing them, and the core tools and techniques used at each stage.

Table 1: Pipeline Stages, Components, and Tools

Pipeline Stage / Focus	**Components / Files**	**Tools & Techniques**
App ↔ Backend Networking	NetworkClient.kt, SplatApiService.kt, Models.kt	Retrofit, OkHttp, Moshi (multipart video upload and job-polling)
Backend Scaffold (v1)	main.py, checker.py, dependency.py, initial tasks.py	FastAPI, initial Open3D-based cleanup
Early Capture Prototype	AR Foundation scripts	AR Foundation (explored and superseded)
COLMAP Integration	run_pipeline() in tasks.py	COLMAP (feature_extractor, sequential_matcher, mapper, image_undistorter), FastGS training, pipeline stabilization

Pipeline Stage / Focus	Components / Files	Tools & Techniques
APK Builds	SplatStudio.apk commits	Gradle build system
Cleanup Pipeline — Appearance	backend/cleanup/appearance_filters.py	Local-neighborhood color-consistency filtering (SH DC term), quaternion rotation-sanity filter
Quality / Calibration Metrics	backend/metrics.py	COLMAP reprojection-error stats, ArUco-marker physical scale calibration (DLT triangulation), PSNR/SSIM/LPIPS against held-out renders
Compression & Serving	splat-transform call in tasks.py, /api/download	.ply to .sog compression, FileResponse serving

4. Coding / Implementation Details

4.1 App–Backend Networking

The Android application's networking layer (NetworkClient.kt, SplatApiService.kt, Models.kt) was implemented using **Retrofit** and **OkHttp**, with **Moshi** handling JSON serialization. This layer supports multipart video upload to the backend and asynchronous job-status polling so the app can track a reconstruction job's progress without blocking the UI.

4.2 Backend Scaffold

The initial **FastAPI** backend scaffold (main.py, checker.py, dependency.py, and an initial version of tasks.py) established the core service structure, including an initial **Open3D**-based point-cloud cleanup step, later superseded by the custom appearance-based cleanup module described below.

4.3 Early Capture Prototype

An early capture prototype was explored using **AR Foundation**. This approach was ultimately superseded in favor of a more direct video-capture-based flow better suited to the **COLMAP**/**FastGS** pipeline.

4.4 COLMAP Integration and Pipeline Stabilization

The core reconstruction pipeline is implemented in run_pipeline() within tasks.py. This function drives **COLMAP**'s feature_extractor, sequential_matcher, mapper, and image_undistorter stages to recover camera poses and undistorted images from the captured video frames, followed by **FastGS** training to produce the initial **Gaussian Splat**. Substantial effort went into stabilizing this pipeline against failures on real-world, non-ideal captures.

4.5 APK Builds

The Android application was packaged and iterated on as **SplatStudio**.apk using the **Gradle** build system, with incremental builds tracked across the internship.

4.6 Cleanup Pipeline — Appearance Filtering

backend/cleanup/appearance_filters.py implements the core cleanup logic used to remove visually inconsistent Gaussians from the trained splat. This includes a local-neighborhood color-consistency filter operating on the spherical harmonics DC (base color) term — flagging Gaussians whose color diverges significantly from their spatial neighborhood — and a quaternion rotation-sanity filter that removes Gaussians with degenerate or non-physical orientation values. Together these filters target the floating and comet-tail-like streak artifacts common in raw splat reconstructions.

4.7 Quality and Calibration Metrics

backend/metrics.py implements quantitative evaluation of each reconstruction. **COLMAP** reprojection-error statistics give a direct measure of camera pose accuracy. Physical scale calibration is performed using **ArUco** markers placed in the scene, with their known physical size used in a Direct Linear Transformation (DLT) triangulation step to recover a metric scale factor for the otherwise scale-ambiguous reconstruction. Reconstruction fidelity is further assessed using **PSNR**, **SSIM**, and **LPIPS** computed against held-out reference renders.

4.8 Compression and Serving

The final cleaned splat is compressed from the raw .ply format to the more compact .sog format via a splat-transform call within tasks.py, and served to the Android client through a FileResponse-backed /api/download endpoint for on-device rendering.

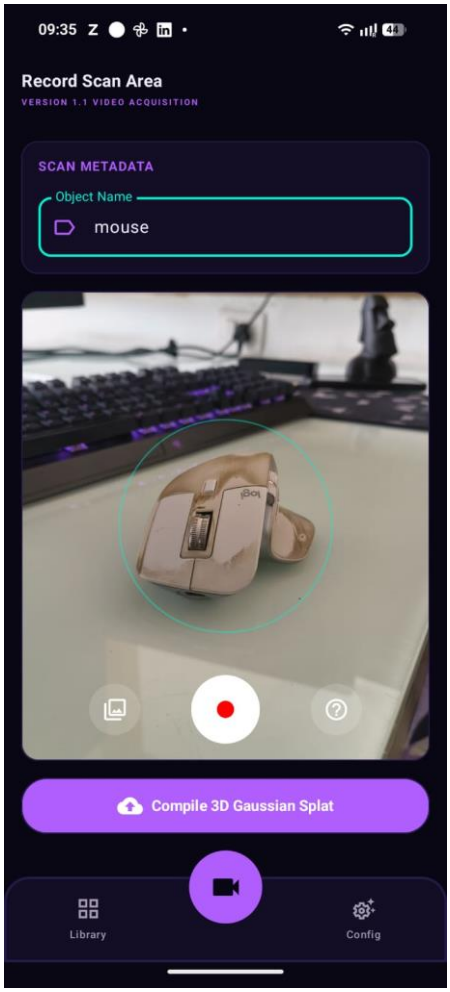

*Figure 2: **SplatStudio** Android App – Capture Screen*

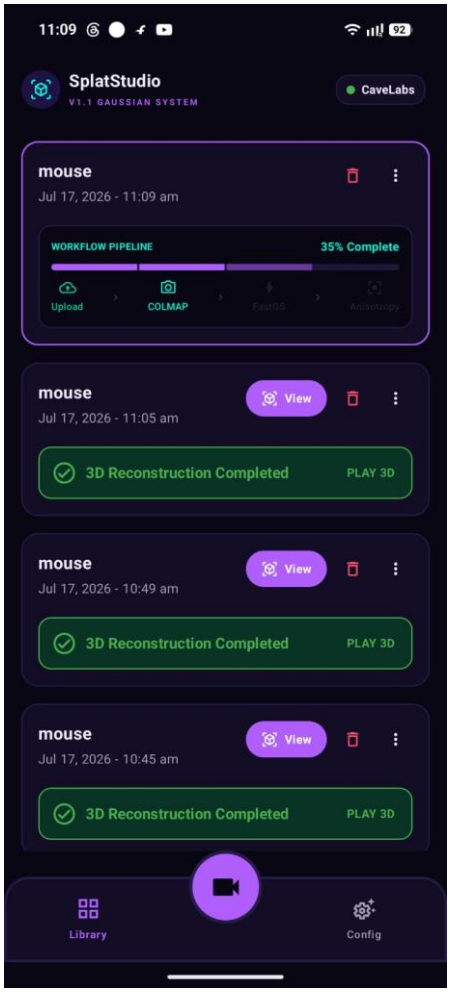

*Figure 3: **SplatStudio** Android App – Progress Bar*

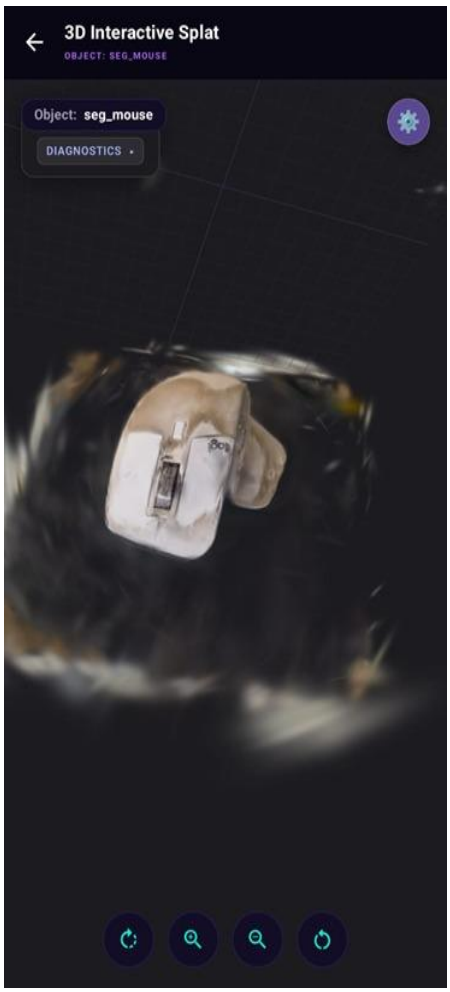

*Figure 4: **SplatStudio** Android App – Rendered View*

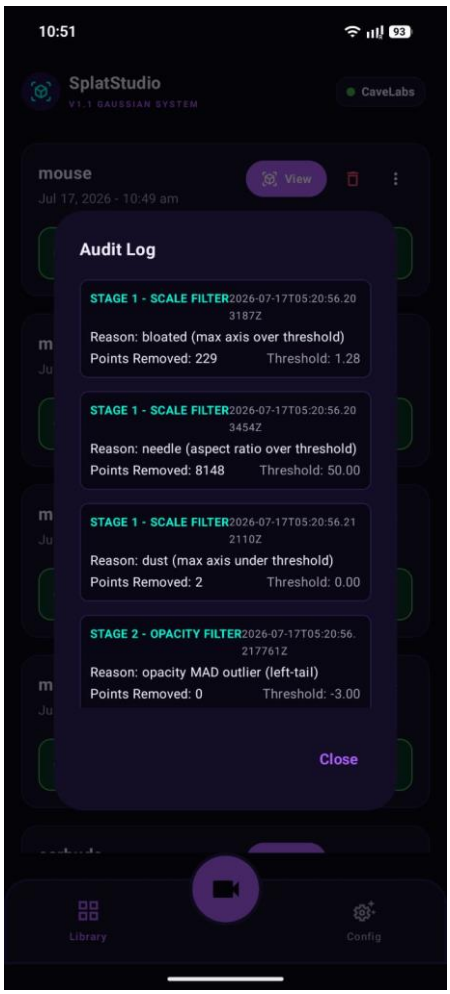

*Figure 5: **SplatStudio** Android App – Audit Log*

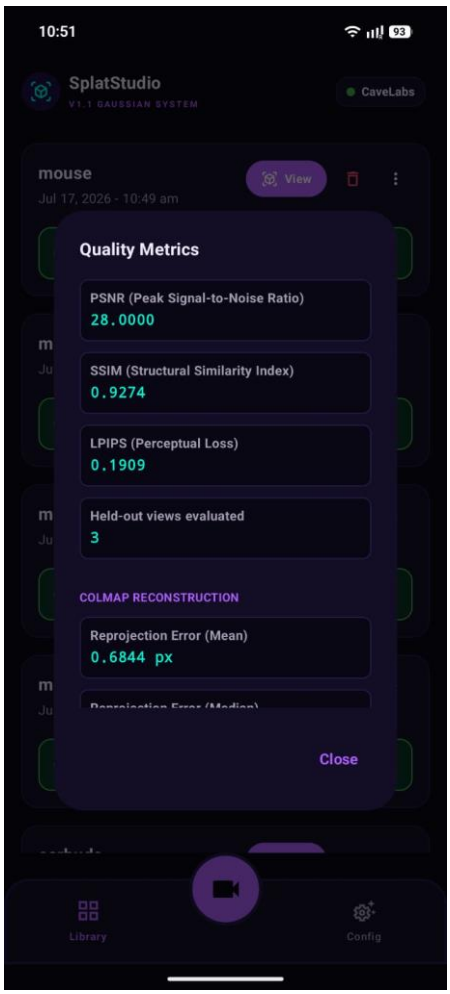

*Figure 6: **SplatStudio** Android App – Metrics*

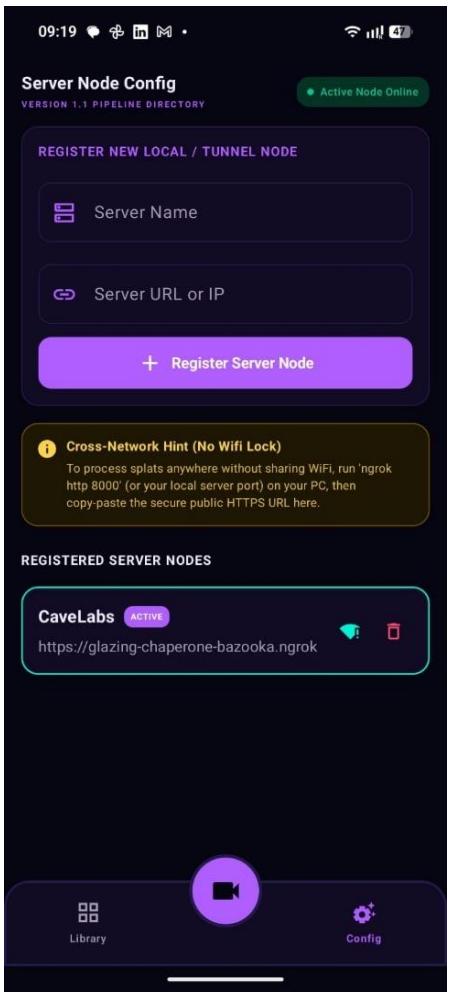

*Figure 7: **SplatStudio** Android App – Server Page*

5. Project Results / Learning Outcomes

The most significant technical learning from this internship was a working understanding of **Structure-from-Motion** — how camera poses and sparse scene geometry can be recovered purely from unordered images via feature extraction and matching — and, more broadly, what it takes to turn a research pipeline (**COLMAP**, **FastGS**) into a real, callable backend service that has to handle imperfect, real-world captures rather than clean benchmark data.

The cleanup stage of the pipeline was a recurring source of difficult bugs — reconstructions would train successfully but still show floating and streak-like Gaussian artifacts that were not obviously explained by any single stage of the pipeline. Isolating whether these artifacts originated from camera pose errors, training instability, or the cleanup filters themselves required systematically testing each stage in isolation. These issues were worked through as a team rather than individually, and were eventually resolved through the combination of the local-neighborhood color-consistency and quaternion rotation-sanity filters described in Section 4.6.

Beyond the technical pipeline itself, the internship reinforced how much easier collaborative work becomes when responsibilities are clearly divided along the pipeline's natural stages, with each person able to go deep on their component while still being able to reason about how it fits into the whole system — a lesson that carried over into how the author now thinks about system design more broadly.

6. Conclusion

By the end of the eight-week internship, the project's full pipeline — video capture, **COLMAP**-based pose recovery, **FastGS** training, appearance-based cleanup, quality and calibration metrics, and compressed on-device serving — was successfully running end-to-end. A handheld video capture could be taken through the Android app and result in a cleaned, quantitatively assessed, compressed **Gaussian Splat** rendered back on-device, closing the loop the project set out to build.

While the foundational pipeline is functionally robust, there are several avenues for future enhancement to elevate the system to enterprise-grade forensic standards:

- Semantic Object Segmentation: While current object isolation relies on spatial heuristics (**RANSAC** and **DBSCAN**), future iterations could integrate lightweight visual foundation models (e.g., **Segment Anything**) to provide semantic, object-aware tagging of specific evidence without requiring manual threshold tuning.

- Dynamic Threshold Optimization: The adaptive MAD-based filtering could be further augmented with a machine-learning feedback loop, allowing the backend to dynamically adjust geometric outlier thresholds based on lighting and texture conditions extracted from the initial video frames.

- Enhanced On-Device Rendering: Further optimization of the Android viewer is planned, including implementing level-of-detail (LOD) degradation and tiled rendering, to support smooth navigation of dense splats on lower-end mobile devices.

On a personal level, the internship reinforced the author's interest in AR/VR as a career direction, with a growing pull specifically toward system design — building and reasoning about how multiple components (capture, backend processing, storage, on-device rendering) fit together as a whole system — rather than any single stage in isolation.

7. References / Bibliography

[1] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, "**3D Gaussian Splat**tting for Real-Time Radiance Field Rendering," ACM Transactions on Graphics, vol. 42, no. 4, July 2023.

[2] J. L. Schönberger and J.-M. Frahm, "Structure-from-Motion Revisited," IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016.

[3] J. L. Schönberger, E. Zheng, M. Pollefeys, and J.-M. Frahm, "Pixelwise View Selection for Unstructured Multi-View Stereo," European Conference on Computer Vision (ECCV), 2016.

[4] FastAPI Documentation, https://fastapi.tiangolo.com/

[5] COLMAP Documentation, https://colmap.github.io/

[6] Open3D: A Modern Library for 3D Data Processing, http://www.open3d.org/